# 数值线性代数　第 19 讲：最小二乘算法的稳定性

Lloyd N. Trefethen、David Bau III　中文阅读版

正如第 11 讲所述，最小二乘问题可以用多种方法求解，包括正规方程、Householder 三角化、Gram–Schmidt 正交化和 SVD. 本讲比较这些方法，并说明使用正规方程通常是不稳定的.

## 例子

为了说明各种算法的行为，考虑 $m = 100, n = 15$ 的数值例子. 下面是 MATLAB 设置:

```
m = 100; n = 15;
t = (0:m-1)'/(m-1);                Set t to a discretization of [0, 1].
A = []; for i=1:n,                  Construct Vandermonde matrix.
  A = [A t.^(i-1)]; end
b = exp(sin(4*t));                 Right-hand side.
b = b/2006.787453080206;            Normalization (see text).
```

这个例子的思想，是在区间 $[0, 1]$ 上用 14 次多项式对函数 $\exp(\sin(4t))$ 做最小二乘拟合. 首先将 $[0, 1]$ 离散化，定义由 100 个等距点组成的向量 $t$. 矩阵 $A$ 是 $100 \times 15$ 的 Vandermonde 矩阵，其列是在这些点处采样的 $1, t, \ldots, t^{14}$；右端 $b$ 是在同一点采样的函数 $\exp(\sin(4t))$.

代码最后一行看起来很奇怪，原因如下. 为简单起见，我们只比较不同算法计算出的系数 $x_{15}$. 如果没有最后一行，$x_{15}$ 的正确值是

$$2006.787453080206\ldots,$$

这个数由扩展精度算术包得到. 将 $b$ 除以它之后，问题的解满足 $x_{15} = 1$，比较就容易得多.

为了解释观察结果，需要使用式 (18.3)–(18.5) 中的量. 可以用 MATLAB 的反斜杠运算符数值求解最小二乘问题，从而得到足够准确的估计:

```
x = A\b; y = A*x;                  Solve least squares problem.
kappa = cond(A)
```

```
    kappa = 2.2718e+10                    kappa(A)
theta = asin(norm(b-y)/norm(b))
    theta = 3.7461e-06                    theta
eta = norm(A)*norm(x)/norm(y)
    eta = 2.1036e+05                      eta
```

结果 $\kappa(A) \approx 10^{10}$ 表明单项式 $1, t, \ldots, t^{14}$ 构成了一个高度病态的基. 结果 $\theta \approx 10^{-6}$ 表明 $\exp(\sin(4t))$ 可以被 14 次多项式非常精确地拟合. 拟合如此接近，以至于为避免消去误差，我们用

$$\theta = \sin^{-1}\left(\frac{\|b-y\|}{\|b\|}\right)$$

而不是式 (18.4) 来计算 $\theta$. 至于 $\eta$，它约为 $10^5$，大致位于式 (18.6) 允许的两个极端值 1 和 $\kappa(A)$ 的中间.

将这些数代入定理 18.1，可知在本例中，$b$ 和 $A$ 的扰动对 $y$ 和 $x$ 的条件数近似为

| | $y$ | $x$ |
|---|---|---|
| $b$ | $1.0$ | $1.1 \times 10^5$ |
| $A$ | $2.3 \times 10^{10}$ | $3.2 \times 10^{10}$ |

# Householder 三角化

正如第 11 讲所述，求解最小二乘问题的标准算法，是通过 Householder 三角化进行 QR 分解（算法 11.2）. 下面是 MATLAB 实验：

```
[Q,R] = qr(A,0);                      Householder triang. of A.
x = R\(Q'*b);                         Solve for x.
x(15)
    ans = 1.00000031528723
```

如何理解这个结果? 由于我们做了归一化，正确答案应为 $x_{15} = 1$，所以相对误差约为 $3 \times 10^{-7}$. 计算使用 IEEE 双精度算术，$\varepsilon_{\text{machine}} \approx 10^{-16}$，这意味着舍入误差被放大了约 $10^9$ 倍. 乍看之下这似乎很糟，但上面的表提醒我们，$A$ 的扰动对 $x$ 的条件数约为 $10^{10}$. 因此，$x_{15}$ 的不准确完全可以由病态性解释，而不是由不稳定性解释. 算法 11.2 看起来是后向稳定的.

上面显式形成了 $Q$，但正如第 10、16 讲强调的，这并非必要. 只需存储算法 10.1 每一步确定的向量 $v_k$（式 (10.5)），再用算法 10.2 计算 $Q^*b$. 在 MATLAB 中，可以不只对 $A$ 做 QR 分解，而是对 $m \times (n+1)$ 的增广矩阵 $[A\ b]$ 做 QR 分解. 在这个过程中，使 $A$ 变为上三角的 $n$ 个 Householder 反射器也会作用于 $b$，把向量 $Q^*b$ 留在第 $n+1$ 列的前 $n$ 个位置. 随后应用第 $n+1$ 个反射器，将该列的第 $n+2$ 到第 $m$ 个条目变为零；这不会改变我们关心的前 $n$ 个条目. 因此：

```
[Q2,R2] = qr([A b],0);              Householder triang. of [A b].
R2 = R2(1:n,1:n);                   Extract R ...
Qb = R2(1:n,n+1);                   ... and Q'*b.
x = R2\Qb;                          Solve for x.
x(15)
    ans = 1.00000031529465
```

这个答案与前一个几乎相同，说明 QR 分解 $A$ 引入的误差淹没了计算 $Q^*b$ 引入的误差.

还有第三种通过 Householder 三角化求解最小二乘问题的 MATLAB 方法：使用内置反斜杠运算符，正如前面计算 $\kappa(A)$、$\theta$ 和 $\eta$ 时所做的那样：

```
x = A\b;                            Solve for x.
x(15)
    ans = 0.99999994311087
```

这个结果明显不同，而且准确了一个数量级. 原因是 MATLAB 的反斜杠运算符使用带列选主元的 QR 分解，其形式为 $AP = QR$，其中 $P$ 是置换矩阵. 本书不讨论列选主元.

从范数级稳定性分析的角度看，这三种 QR 分解变体是等价的；可以证明它们全部后向稳定.

**定理 19.1.** 设满秩最小二乘问题 (11.2) 在满足公理 (13.5)、(13.7) 的计算机上通过 Householder 三角化（算法 11.2）求解. 则该算法后向稳定，即计算得到的解 $\widetilde{x}$ 满足

$$\|(A+\delta A)\widetilde{x} - b\| = \min, \qquad \frac{\|\delta A\|}{\|A\|} = O(\varepsilon_{\text{machine}}), \tag{19.1}$$

其中 $\delta A \in \mathbb{C}^{m\times n}$. 无论 $Q^*b$ 是通过显式形成 $Q$ 计算，还是由算法 10.2 隐式计算，结论都成立；对任意列选主元的 Householder 三角化也成立.

# Gram–Schmidt 正交化

求解最小二乘问题的另一种方法是改进的 Gram–Schmidt 正交化（算法 8.1）. 当 $m \approx n$ 时，它比 Householder 方法需要稍多的运算；但当 $m \gg n$ 时，两种算法的运算量都渐近为 $2mn^2$.

下面的 MATLAB 序列以直接方式实现该算法. 函数 'mgs' 是算法 8.1 的一个实现（此处不展示），与第 9 讲实验 2 中的实现相同：

```
[Q,R] = mgs(A);                     Gram-Schmidt orthog. of A.
x = R\(Q'*b);                       Solve for x.
x(15)
    ans = 1.02926594532672
```

这个结果很差. 舍入误差被放大了约 $10^{14}$ 倍，远大于问题的条件数. 事实上，该算法是不稳定的，原因很容易找出：正如第 9 讲末尾所述，Gram–Schmidt 正交化通常产生列并不准确正交的矩阵 $\widetilde{Q}$. 上面的算法依赖这种正交性，因此会受到相应影响.

可以通过改写算法避免不稳定性. 即使 $\widetilde{Q}$ 的列不准确正交，Gram–Schmidt 迭代仍会给出准确的乘积 $\widetilde{Q}\widetilde{R}$. 一种办法是为向量 $\widetilde{R}x$ 建立正规方程

$$\widetilde{R}x = (\widetilde{Q}^*\widetilde{Q})^{-1}\widetilde{Q}^*b,$$

然后回代求 $x$. 只要计算得到的 $\widetilde{Q}$ 至少是良态的，这种方法就不会遭受下面讨论的、将正规方程应用于任意矩阵时的那些不稳定性. 不过它包含不必要的额外工作，实践中不应使用.

稳定 Gram–Schmidt 方法的更好办法，是像前面的第二个 Householder 实验一样使用增广方程组：

```
[Q2,R2] = mgs([A b]);                Gram-Schmidt orthog. of [A b].
R2 = R2(1:n,1:n);                    Extract R ...
Qb = R2(1:n,n+1);                    ... and Q'*b.
x = R2\Qb;                           Solve for x.
x(15)
    ans = 1.00000005653399
```

现在结果与 Householder 三角化一样好. 可以证明这总是成立.

**定理 19.2.** 满秩最小二乘问题 (11.2) 通过 Gram–Schmidt 正交化求得的解也后向稳定，满足式 (19.1)，前提是像上面的代码片段那样隐式形成 $\widetilde{Q}^*b$.

# 正规方程

求解最小二乘问题的另一种根本不同的方法，是求解正规方程（算法 11.1），通常使用 Cholesky 分解（第 23 讲）. 当 $m \gg n$ 时，这种方法比依赖显式正交化的方法快一倍，渐近只需要 $mn^2$ 次浮点运算（式 (11.14)）. 在下面的实验中，用一行 MATLAB 反斜杠命令求解：

```
x = (A'*A)\(A'*b);                    Form and solve normal equations.
x(15)
    ans = 0.39339069870283
```

这个结果非常糟糕！这是我们得到的最差结果，甚至没有一位正确数字. 显然，使用正规方程求解最小二乘问题是不稳定的. 下面解释这一现象；这正是良态性和稳定性相互作用的典型例子. 另外，正规方程的使用非常普遍，因此理解其中的风险十分重要.

假设我们有一个求解满秩问题 (11.2) 的后向稳定算法，得到的解 $\widetilde{x}$ 满足

$$\|(A+\delta A)\widetilde{x}-b\|=\min, \qquad \frac{\|\delta A\|}{\|A\|}=O(\varepsilon_{\text{machine}}).$$

（允许 $b$ 也发生扰动，或者考虑稳定性而非后向稳定性，都不会改变主要结论.）由定理 15.1 和 18.1，有

$$\frac{\|\widetilde{x}-x\|}{\|x\|}=O\left(\left(\kappa+\frac{\kappa^2\tan\theta}{\eta}\right)\varepsilon_{\text{machine}}\right), \qquad \kappa=\kappa(A). \tag{19.2}$$

现在假设 $A$ 病态，即 $\kappa\gg 1$，且 $\theta$ 远离 $\pi/2$. 根据不同参数的取值，可能出现两种完全不同的情形. 如果 $\tan\theta$ 是 1 的量级（即拟合并不特别接近），且 $\eta\ll\kappa$，则式 (19.2) 右端为 $O(\kappa^2\varepsilon_{\text{machine}})$. 另一方面，如果 $\tan\theta$ 接近零（拟合非常接近），或者 $\eta$ 接近 $\kappa$，则界为 $O(\kappa\varepsilon_{\text{machine}})$. 最小二乘问题的条件数可能处于从 $\kappa$ 到 $\kappa^2$ 的任意位置.

现在考察用正规方程求解 (11.2) 的情形：

$$(A^*A)x=A^*b.$$

Cholesky 分解对这个方程组是稳定算法，即它得到的解 $\widetilde{x}$ 满足

$$(A^*A+\delta H)\widetilde{x}=A^*b, \qquad \frac{\|\delta H\|}{\|A^*A\|}=O(\varepsilon_{\text{machine}})$$

（定理 23.3）. 但是矩阵 $A^*A$ 的条件数是 $\kappa^2$，而不是 $\kappa$. 因此正规方程所能期待的最好结果是

$$\frac{\|\widetilde{x}-x\|}{\|x\|}=O(\kappa^2\varepsilon_{\text{machine}}). \tag{19.3}$$

正规方程的行为由 $\kappa^2$ 而不是 $\kappa$ 控制.

结论已经很清楚. 如果 $\tan\theta$ 为 1 的量级且 $\eta\ll\kappa$，或者 $\kappa$ 为 1 的量级，那么式 (19.2) 和 (19.3) 同阶，正规方程是稳定的. 但如果 $\kappa$ 很大，且 $\tan\theta$ 接近零或 $\eta$ 接近 $\kappa$，式 (19.3) 就会远大于式 (19.2)，正规方程是不稳定的. 对于包含很接近拟合的病态问题，正规方程通常是不稳定的. 在本例中，$\kappa^2\approx 10^{20}$，因此 Cholesky 分解得不到任何正确数字并不令人意外.

按我们的定义，只有在所考虑的全部问题上都一致表现良好的算法才是稳定的. 因此，可以用下面的结果把刚才的观察正式化.

**定理 19.3.** 满秩最小二乘问题 (11.2) 通过正规方程（算法 11.1）得到的解是不稳定的. 不过，如果把问题限制在如下某一类中，则可以获得稳定性：$\kappa(A)$ 有统一的上界，或者 $(\tan\theta)/\eta$ 有统一的正下界.

# SVD

第 11 讲还提到过一种最小二乘算法：使用 SVD（算法 11.3）. 像大多数基于 SVD 的计算一样，这个方法是稳定的：

```
[U,S,V] = svd(A,0);                    Reduced SVD of A.
x = V*(S\(U'*b));                      Solve for x.
x(15)
    ans = 0.99999998230471
```

事实上，这是所有实验结果中最准确的，比带列选主元的 Householder 三角化（MATLAB 的反斜杠）还准确约 3 倍. 可以证明通常形式的定理.

**定理 19.4.** 满秩最小二乘问题 (11.2) 通过 SVD（算法 11.3）得到的解后向稳定，满足估计式 (19.1).

# 秩亏最小二乘问题

本讲确定了四种用于满秩线性最小二乘问题的后向稳定算法：Householder 三角化、带列选主元的 Householder 三角化、隐式计算 $\widetilde{Q}^*b$ 的改进 Gram–Schmidt，以及 SVD. 从满秩问题 (11.2) 的经典范数级稳定性分析角度看，这些算法之间的差异很小，因此不妨采用最简单、最便宜的不带列选主元的 Householder 三角化.

不过，还有其他类型的最小二乘问题，其中列选主元和 SVD 具有特殊重要性. 这类问题满足 $\operatorname{rank}(A) < n$，甚至可能有 $m < n$，因而方程组是不定的. 除非增加额外条件（通常要求 $x$ 的范数尽可能小），否则这类问题没有唯一解. 另一个困难是，正确解依赖于 $A$ 的秩，而在存在舍入误差时数值确定秩从来都不是小事.

因此，秩亏最小二乘问题不是最小二乘问题的一个简单子类，而是根本不同的问题. 既然解的定义已经改变，就没有理由认为对满秩问题稳定的算法在秩亏情形也稳定. 事实上，对秩亏问题完全稳定的算法只有基于 SVD 的算法. 另一种选择是带列选主元的 Householder 三角化，它对几乎所有问题都稳定. 本书不展开细节.

# 习题

**19.1** 设 $A \in \mathbb{C}^{m\times n}$ 满秩（秩为 $n$），$b \in \mathbb{C}^m$. 考虑块 $2\times 2$ 方程组

$$\begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \tag{19.4}$$

其中 $I$ 是 $m\times m$ 单位矩阵. 证明这个系统有唯一解 $(r,x)^T$，并证明向量 $r$ 和 $x$ 分别是最小二乘问题 (18.1) 的残差和解.

**19.2** 下面是 MATLAB 某个内置 m 文件的精简版本：

```
[U,S,V] = svd(A);
S = diag(S);
```

```
tol = max(size(A))*S(1)*eps;
r = sum(S > tol);
S = diag(ones(r,1)./S(1:r));
X = V(:,1:r)*S*U(:,1:r)';
```

这个程序计算什么?